HFWU NÜRTINGEN-GEISLINGEN · FAKULTÄT WIRTSCHAFT & GESELLSCHAFT · PROF. MATHIAS ENGEL

Trinity Agent Architektur

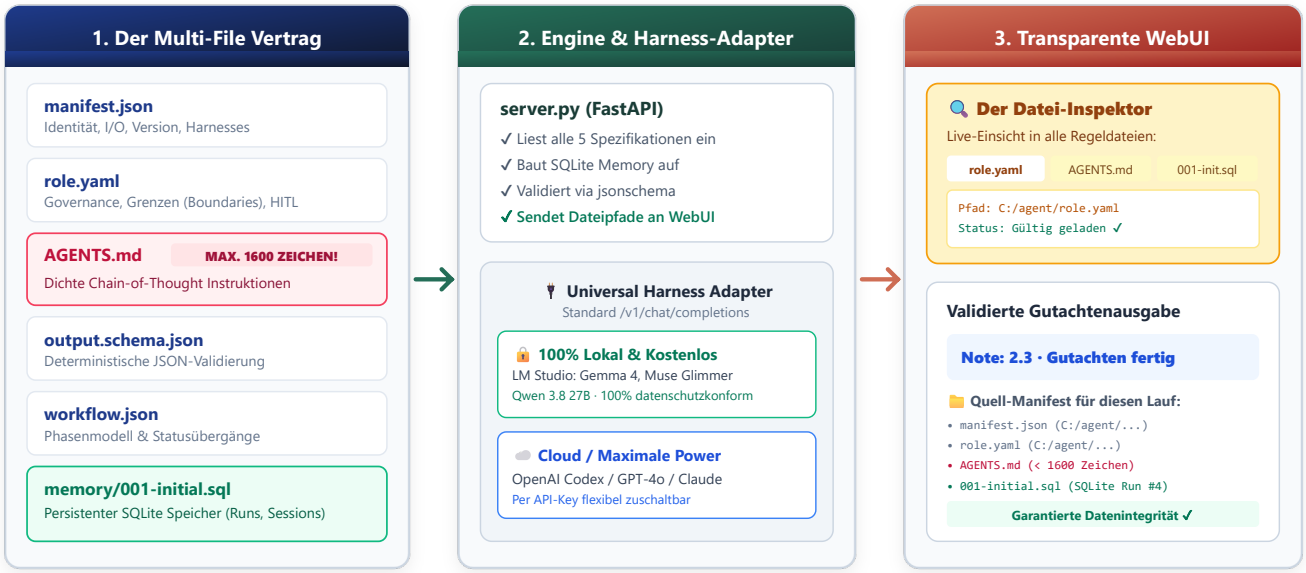
Autarker Praxisleitfaden & Master-Prompt für Studierende

Thema: Vertragsbasierte KI-Agenten, SQLite Memory & WebUI-Transparenz

Wintersemester 2026/27 · Stand: Oktober 2026

📊 Infografik: Was zeichnet einen Trinity Agenten aus?

Architekturüberblick



1. Einleitung: Was entsteht hier und was macht diesen Ansatz besonders?

Klassische KI-Anwendungen im Studium beginnen meist mit einem langen Chat-Prompt in Web-Interfaces wie ChatGPT oder Claude. Für einfache Fragen genügt das. Sobald jedoch reale, automatisierbare Geschäftsprozesse abgebildet werden sollen, scheitern solche monolithischen „Mega-Prompts“.

> 💡 **Das didaktische Kernprinzip für Studierende:**
> Mit diesem Leitfaden baust du keinen simplen Chatbot, sondern ein vollwertiges, industrietaugliches **KI-Agenten-System**. Das System läuft **100% autark** auf deinem eigenen Rechner (ohne Abhängigkeit von einem zentralen Server), schützt durch lokale LLM-Unterstützung vertrauliche Unternehmensdaten und beweist durch den integrierten **Datei-Inspektor** bei jeder Ausführung, aus welcher Datei welche Entscheidung abgeleitet wurde.

Die beiden goldenen Architektur-Regeln

REGEL 1: SYSTEM-PROMPT SCHLANK HALTEN

Die Datei `AGENTS.md` darf maximal 1600 Zeichen umfassen!

Ein häufiger Fehler von KI-Einsteigern sind riesige, unübersichtliche Systemprompts („Prompt-Spaghetti“). Im Trinity-Standard erzwingen wir maximale Prägnanz: Dichte Chain-of-Thought-Instruktionen, klare Kriterien und kein Fülltext. Alle Schranken gehören in `role.yaml`, die Datenstrukturen in `output.schema.json`!

REGEL 2: PERSISTENTES GEDÄCHTNIS MIT SQL

Jeder Agent besitzt ein SQLite-Gedächtnis via `memory/migrations/001-initial.sql`

Ein professioneller Agent darf nicht bei jedem Neustart alles vergessen. Die SQL-Migration definiert Tabellen für Sitzungen (`sessions`), Auswertungen und Kernwissen (`memories`). Das Backend initialisiert die Datenbank beim Start automatisch.

Die 6 Säulen des Trinity-Dateivertrags

Datei	Format	Zweck & Bedeutung
`manifest.json`	JSON	**Visitenkarte & Metadaten:** ID, Name, Version, Inputs, Outputs, Memory-Pfad und erlaubte LLM-Harnesses.
`role.yaml`	YAML	**Governance & Schranken:** Persona, Leitbild, strikte Verbote (*Boundaries*), Qualitätsprüfungen (*Quality Gates*) und Mensch-im-Loop-Freigaben (*Human Gates*).
`AGENTS.md`	Markdown	**Operatives Handbuch:** Chain-of-Thought-Denkschritte & Richtlinien. **Strikt limitiert auf max. 1600 Zeichen!**
`output.schema.json`	JSON-Schema	**Garantierte Ausgabestruktur:** Erzwingt maschinenlesbare Pflichtfelder (Note, Kriterienmatrix, Stärken, Schwächen, Detailgutachten).
`workflow.json`	JSON	**Phasenmodell:** Definiert die linearen Bearbeitungsphasen (z. B. Input → Audit → Synthese → Urteil).
`memory/migrations/001-initial.sql`	SQL (SQLite)	**Persistentes Gedächtnis:** Schema für Sitzungen, Läufe und Memory-Einträge.

2. Der universelle Master-Prompt für KI-Coding-Assistenten

Kopiere den folgenden Textblock vollständig und füge ihn in ein KI-Entwicklungstool deiner Wahl ein (z. B. **Antigravity**, **OpenAI Codex / ChatGPT**, **Claude** oder **Cursor**). Die KI wird dir auf Knopfdruck das vollständige, lauffähige Projekt erstellen.

```
Du bist ein erfahrener KI-Architekt und sollst für mich einen vollständigen, autarken "Trinity Agenten" entwickeln.
Der Agent soll dem Trinity-Architekturstandard entsprechen, jedoch völlig unabhängig von einem zentralen Hub laufen. Er soll über ein Harness (OpenAI-kompatible Schnittstelle wie Codex, GPT-Modelle, LM Studio oder Ollama) ausführbar sein, ein persistentes SQLite-Gedächtnis besitzen und über eine einfache WebUI mit Datei-Inspektor bedient werden können.

### 1. Die didaktischen Kernprinzipien & harten Regeln
- Im Trinity-Standard ist ein Agent kein monolithischer Skriptcode, sondern ein definierter Vertrag aus mehreren modularen Dateien.
- HARTE ARCHITEKTUR-REGEL: Die Datei AGENTS.md darf MAXIMAL 1600 ZEICHEN umfassen! Halte sie prägnant (Chain-of-Thought, Kriterien). Alle Schranken gehören in role.yaml!
- SQLITE GEDÄCHTNIS: Der Agent besitzt ein persistentes SQLite-Schema unter memory/migrations/001-initial.sql für Sitzungen, Auswertungen und Erinnerungen.
- DATEIPFAD-TRANSPARENZ: Sowohl in der WebUI als auch bei der Ausführung MUSS transparent angezeigt werden, welche Dateien existieren, wo sie im Dateisystem liegen und was ihr Inhalt ist.

### 2. Projektstruktur
Erstelle folgendes Projektverzeichnis:
my-trinity-agent/
├── manifest.json                  # Metadaten, Fähigkeiten, I/O-Definition & Memory-Pfad
├── role.yaml                      # Persona, Purpose, Grenzen (Boundaries), Quality Gates & HITL
├── AGENTS.md                      # Detaillierte Systeminstruktionen (STRIKT MAX 1600 ZEICHEN!)
├── output.schema.json             # JSON-Schema für validierte, strukturierte Ausgaben
├── workflow.json                  # Phasenmodell des Agenten (z. B. Input -> Analyse -> Urteil)
├── memory/
│   └── migrations/
│       └── 001-initial.sql        # SQLite-Schema für sessions, evaluations und memories
├── server.py                      # Autarker Server (FastAPI + SQLite) mit OpenAI/Codex-Harness
```

```
└── static/
    └── index.html                  # Single-Page-WebUI mit Datei-Inspektor und Ausführungsmaske

### 3. Beispiel-Agent: "Business Case & Methodik-Evaluator"
Nutze als konkreten Beispiel-Agenten einen akademischen Evaluator:
- Zweck: Begutachtet Seminararbeiten, Business Pläne und Fallstudien auf methodische Stringenz, logische Konsiste
nz, Wirtschaftlichkeit und Qualität der Quellen.
- Boundaries: Ausschließlich objektive Kritik; Bewertung nach Drittelnoten (1.0–5.0); Kennzeichnung von Annahmen
ohne Datenbasis.
- Quality Gates: Prüfung auf Plausibilität von Zahlen und saubere Problem-Lösungs-Kausalität.

### 4. Das Backend (server.py mit FastAPI und SQLite)
Erstelle ein lauffähiges Python-Backend, das:
1. Beim Start die SQLite-Datenbank agent_memory.sqlite3 mit 001-initial.sql initialisiert.
2. Die lokalen Dateien beim Start einliest und deren absolute Dateipfade ermittelt.
3. Einen Endpoint GET /api/agent/files bereitstellt, der alle Dateien inklusive Pfaden und Inhalten an die WebUI
liefert (für den Datei-Inspektor).
4. Einen Endpoint POST /api/agent/run bereitstellt, der:
   - Die Nutzereingabe empfängt.
   - Den System-Prompt dynamisch aus role.yaml und AGENTS.md kompiliert.
   - Eine konfigurierbare OpenAI-/Codex-Schnittstelle aufruft (/v1/chat/completions – kompatibel mit OpenAI, LM S
tudio auf http://localhost:1234/v1 oder Ollama auf http://localhost:11434/v1).
   - Die Antwort gegen output.schema.json validiert.
   - Das Ergebnis dauerhaft in SQLite (sessions und evaluations) speichert.
   - Im Response-Objekt und in der Konsole explizit die Liste aller verwendeten Quell-Dateien und deren Pfade mit
zurückgibt!

### 5. Die WebUI (static/index.html)
Erstelle eine moderne, responsive Single-Page-Anwendung:
1. Header: Zeigt Agenten-Name, Version und Status aus manifest.json.
2. Harness-Konfiguration: Eingabefelder für Base-URL (z. B. http://localhost:1234/v1), Modellname und optional AP
I-Key.
3. Datei-Inspektor: Tabs, die alle Agenten-Dateien (inkl. 001-initial.sql) anzeigen. Klickt man auf eine Datei, s
ieht man ihren echten Pfad auf der Festplatte und ihren Inhalt.
4. Eingabebereich: Textfeld für den zu evaluierenden Text und ein "Agent ausführen"-Button.
5. Ergebnisanzeige: Zeigt Note, TL;DR, Kriterien-Tabelle, Gutachten und einen Kasten "Verwendete Agenten-Dateien"
mit den genauen Pfaden.

Gib mir den kompletten Code für alle Dateien und eine kurze Startanleitung.
```

3. Die Steuerdateien des Referenz-Agenten im Detail

A. `manifest.json`

```
{
  "schemaVersion": "1.0",
  "id": "business-case-evaluator",
  "name": "Business Case & Methodik-Evaluator",
  "shortName": "CaseEvaluator",
  "version": "1.0.0",
  "description": "Autarker Trinity-Agent zur akademischen Begutachtung von Business Cases.",
  "category": "Hochschule & Analyse",
  "capabilities": [
    "Methoden- und Konsistenzprüfung",
    "Finanz- und Plausibilitäts-Check",
    "Stärken-/Schwächen-Profilierung",
    "Strukturiertes Gutachten nach Kriterienmatrix"
  ],
  "harnesses": ["codex", "chatgpt", "lmstudio", "ollama"],
  "memory": {
    "schemaVersion": 1,
    "migration": "memory/migrations/001-initial.sql"
  },
  "specFiles": {
    "manifest": "manifest.json",
    "role": "role.yaml",
    "instructions": "AGENTS.md",
```

```
    "outputSchema": "output.schema.json",
    "workflow": "workflow.json",
    "memorySql": "memory/migrations/001-initial.sql"
  }
}
```

B. `role.yaml`

```
schema_version: 1
role: CaseEvaluator
purpose: Wissenschaftliche und strategische Evaluation von Business Cases und Projektberichten.
boundaries:
  - Bewertet neutral und sachlich; keine Schönfärberei.
  - Notenskala 1.0 bis 5.0 (ausschließlich offizielle Drittelnoten: 1.0, 1.3, 1.7, 2.0, 2.3, 2.7, 3.0, 3.3, 3.7,
4.0, 5.0).
  - Unbelegte Annahmen in Business Plänen werden explizit als methodisches Risiko markiert.
quality_gates:
  - Prüfung der Kausalitätskette: Problem -> Marktanalyse -> Lösung -> Monetarisierung.
  - Plausibilitäts-Check der Kernkennzahlen (CAC, CLV, Deckungsbeiträge, Break-Even).
human_gates:
  - Noten unter 4.0 (Nicht bestanden) oder 1.0 (Sehr gut) erfordern Dozenten-Freigabe.
```

C. `AGENTS.md` **(Exakt 842 Zeichen – Limit: max. 1600!)**

```
# Business Case Evaluator – System-Instruktionen

## Ziel
Erstelle ein fundiertes, detailliertes Gutachten für den eingereichten Business Case / Seminararbeit.

## Methodisches Vorgehen
1. Plausibilitätsanalyse: Prüfe Marktlücke, Zielgruppendefinition und Kernnutzenversprechen.
2. Methodik-Audit: Sind die getroffenen Annahmen mit validen Quellen belegt?
3. Kriterienbewertung (jeweils 25% Gewichtung):
   - Problem & Nutzenversprechen (25%)
   - Marktanalyse & Wettbewerbsdifferenzierung (25%)
   - Finanzmodell, Skalierbarkeit & Plausibilität (25%)
   - Wissenschaftlicher Aufbau, Stil & Quellenqualität (25%)
4. Notenurteil: Berechne die Gesamtnote und begründe sie analytisch.

## Ausgabeformat
Antworte AUSSCHLIESSLICH mit einem validen JSON-Objekt, das exakt dem output.schema.json entspricht.
Keine Einleitungstexte, keine Markdown-Codeblöcke um das JSON.
```

D. `memory/migrations/001-initial.sql`

```
-- Persistentes SQLite-Gedächtnis für Trinity-Agenten
CREATE TABLE IF NOT EXISTS sessions (
  id TEXT PRIMARY KEY,
  title TEXT NOT NULL,
  created_at TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS evaluations (
  id TEXT PRIMARY KEY,
  session_id TEXT NOT NULL,
  user_input TEXT NOT NULL,
  score REAL NOT NULL,
  result_json TEXT NOT NULL,
  created_at TEXT NOT NULL,
  FOREIGN KEY(session_id) REFERENCES sessions(id)
);

CREATE TABLE IF NOT EXISTS memories (
  id TEXT PRIMARY KEY,
  key TEXT NOT NULL,
  value TEXT NOT NULL,
```

```
  created_at TEXT NOT NULL
);
```

E. `output.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "properties": {
    "tldr": {
      "type": "string",
      "description": "Kompaktes Management Summary mit Notenvorschlag"
    },
    "score": {
      "type": "number",
      "description": "Note zwischen 1.0 und 5.0"
    },
    "criteria": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "criterion": { "type": "string" },
          "rating": { "type": "string" },
          "comment": { "type": "string" }
        },
        "required": ["criterion", "rating", "comment"]
      }
    },
    "strengths": {
      "type": "array",
      "items": { "type": "string" }
    },
    "weaknesses": {
      "type": "array",
      "items": { "type": "string" }
    },
    "detailedReview": {
      "type": "string",
      "description": "Vollständiges akademisches Gutachten in Markdown"
    }
  },
  "required": ["tldr", "score", "criteria", "strengths", "weaknesses", "detailedReview"]
}
```

4. Quickstart & Datenschutz mit lokalem LLM

1. **Pakete installieren:** `pip install fastapi uvicorn httpx jsonschema pyyaml pydantic`
2. **Server starten:** `python server.py` (erstellt automatisch die `agent_memory.sqlite3`)
3. **WebUI öffnen:** Rufe im Browser `http://127.0.0.1:8000` auf.

> 🔒 **Kostenlos & 100% Datenschutz mit LM Studio / Ollama:**
> Für sensible Firmendaten oder unveröffentlichte Seminararbeiten: Starte **LM Studio**, lade ein aktuelles Spitzenmodell (z. B. *Gemma 4 26B*, *Gemma 4 12B* bzw. *Gemma 4 E4B*, *Muse Glimmer* oder *Qwen 3.8 27B*) und klicke auf „Start Server“ auf Port `1234`. In der WebUI des Agenten bleibt die Base-URL auf `http://localhost:1234/v1` aktiv. Die Daten verlassen niemals deinen Laptop!